

인지패턴·맥락 기반 자기인식 도구

팀 분담 명세서

5인 팀의 7주 작업 명세 (W1~W7)

역할별 도구·기술 스택 · 주차별 산출물 · 검증 기준 · 의존성

버전 v1 · 학부 7주 MVP · 베타 8~12명

0장. 공통 운영 룰

0.1 팀 구성 한눈에

역할	이름	책임 영역	주 시간 (W4 제외)
P1	(너)	ML + 기획총괄, LLM 파이프라인, 추천 알고리즘, 발표 자료	10~12
P2	팀원 B	백엔드 (Spring Boot, DB, 인증, quota 관리)	9~11
P3	팀원 C	프론트 메인 (Next.js, 입력·추천 화면)	9~11
P4	팀원 D	프론트 시각화 (Recharts, 누적 카드, 그래프)	9~11
P5	팀원 E	데이터·콘텐츠·QA (라벨링 평정, 베타 운영, 인터뷰)	9~11

0.2 공통 도구

도구	용도	비고
GitHub	코드 + 이슈 관리	main / feature/* 브랜치, PR 머지 필수
Notion	문서 + 작업 트래킹	이 명세서를 Notion 페이지로 동기화
Figma	디자인 협업	P3·P4가 공동 편집, 다른 인원도 보기 가능
Discord	팀 sync	화·목 17시 스탠드업 (1인 1줄, 5분 내)
Postman	API 테스트	P2가 컬렉션 관리, 모두 임포트 사용

0.3 일정 원칙

- W3은 1차 데모 마일스톤. 통과 못 하면 W4 신기능 동결.
- W4는 시험주차. 신기능 동결, 안정화·문서화만. 전체 시간 50% 수준.
- W6은 베타 1주 운영. 전원 일일 점검 30분 의무.

- W7은 발표 준비 전용. 신기능 추가 금지.

0.4 작업 등록 룰

- 모든 작업은 GitHub Issue로 등록 후 셀프 할당
- 작업 시작 시 Issue에 'In Progress' 라벨
- 완료 조건은 'Done When' 섹션에 명시된 항목 100% 만족
- PR 머지 시 해당 Issue 자동 닫힘

0.5 리스크 흡수 메커니즘

상황	발동 룰
W3 데모 마일스톤 미충족	W4-W5 합쳐 신기능 동결, 핵심 기능만 안정화
특정 팀원 시험으로 W4 부담 불가	그 사람의 W3 작업을 80% 부담으로 사전 분할
베타 8명 미달 (W5)	지인 추가 모집, 그래도 부족하면 발표에서 '한계' 인정
LLM API quota 일일 소진	ModelRouter Light tier 우선, 사용자 일일 한도 5로 임시 하향
v1 LogReg AUC < 0.65	v0 단독 추천으로 fallback, 발표에서 정직 보고

P1 (너) — ML + 기획총괄

P1.1 역할 정의

P1은 LLM 라벨링 파이프라인, 추천 알고리즘, 학술 검증의 단독 책임자다. 동시에 5명 팀의 sync를 책임지는 기획총괄을 겸한다. 다른 4명의 작업 일정과 의존성을 일일 관리하며, 발표 자료의 핵심 메시지·이론적 정당화를 작성한다.

P1의 산출물 중 하나라도 늦으면 다른 4명이 줄줄이 막힌다. 특히 W2의 FastAPI 골격, W3의 1차 데모, W5의 LogReg 학습이 단일 실패점이다.

P1.2 사용 도구 상세

도구	용도	수준
Python 3.10+	ML 코드, FastAPI 서비스, 데이터 분석	필수
scikit-learn	Logistic Regression, 5-fold CV, AUC 측정	필수
FastAPI	ML 서비스 (라벨링 + 추천 API)	필수
Gemini API (google-generativeai)	LLM 1차 라벨링 + 2차 정밀화	필수
pandas, numpy	데이터 분석, 합성 데이터 생성	필수
matplotlib / seaborn	검증 결과 시각화 (W6)	권장
GitHub	코드 + 이슈	필수
Notion	발표 자료 초안, 일정 트래킹	필수
Google Docs	발표 슬라이드 (협업)	권장

P1.3 주차별 작업

W1 (1주차)

작업 목록 (총 8~9시간)

- ☐ 새 Gemini API 키 발급 (<https://aistudio.google.com/apikey>)
- ☐ 폴더에 .env 셋업 (GEMINI_API_KEY=...)

- ☐ W1 v6 패키지 11개 파일 폴더 구성 검증
- ☐ run_labeling.bat 더블클릭하여 50문장 라벨링 완료
- ☐ labeling_audit_v1.md에서 PASSED/WARNING/FAILED/MISMATCH 카운트 확인
- ☐ WARNING_MISMATCH 4건의 실제 LLM top 패턴 분석
- ☐ labeling_guide_cases.md 새 파일에 5~10 케이스 정리
- ☐ P5에게 seed 문장 + 평정지 + 가이드 전달
- ☐ P1 본인 평정 (50문장 × 6 패턴 × 3단계, 약 2시간)
- ☐ P5 평정 완료 후 κ 3개 계산 ($P1 \leftrightarrow P5$, $LLM \leftrightarrow P1$, $LLM \leftrightarrow P5$)
- ☐ $\kappa < 0.6$ 이면 P5와 30분 불일치 회의
- ☐ validation_report.md v1 작성 (κ 값 + 해석)
- ☐ Kickoff 회의 90분 진행 — 5인 다 같은 페이지 정렬

입력 (받아야 하는 것)

- 기존 산출물 11개 (drills_seed_v5_1.json, prompt v1, seed_sentences_50.json, run_labeling.py v6 + 6개 모듈)
- P5의 50문장 × 6 패턴 평정 결과 (300셀)

산출물 (만들어야 하는 것)

- labeling_results_v1.json (50개 라벨링 결과)
- labeling_audit_v1.md (자동 검수 보고서)
- labeling_guide_cases.md (5~10 어려운 케이스 + 규칙)
- validation_report.md v1 (κ 3개 + 해석)
- Kickoff 회의록

검증 기준 (Done When)

- FAILED 0건 (또는 API quota 문제만)
- WARNING < 5건 (후처리 적용 후)
- $\kappa(P1, P5) \geq 0.6$
- $\kappa(LLM, P1)$, $\kappa(LLM, P5)$ 계산 완료
- Kickoff 회의록에 5명 다음 주 작업 확정

의존성

- P5의 평정 작업 (P5 W1 작업 끝나야 κ 계산 가능)
- Gemini API 무료 한도 (일일 1500건, 50문장 × 1.5 = 75건 → 안전)

위험

- LLM 호출 실패 → ModelRouter가 자동 폴백 (v6에 박혀있음)
- P5 평정 1주 초과 → W2 첫날까지 연장, 안 되면 P1 단독 평정 + 발표에서 한계 인정

W2 (2주차)

작업 목록 (총 10~11시간)

- ☐ FastAPI 프로젝트 초기 셋업 (uvicorn, pydantic 모델 정의)
- ☐ POST /label 엔드포인트: 한 줄 텍스트 입력 → LLM 호출 → JSON 응답
- ☐ POST /recommend 엔드포인트: user_state + drills 입력 → top-1 추천 + 자기 발견 톤 카피
- ☐ drill_recommender_v5.py 모듈을 FastAPI 안에 통합
- ☐ generate_cumulative_card() 함수 추가 (1회·3회·7회 분기 카피)
- ☐ user_baseline 추출 함수 (최소 7일 데이터 누적 후 작동)
- ☐ quota_manager.py 모듈 작성 (사용자별 일일 한도, 분당 rate limit)
- ☐ ModelRouter를 FastAPI 서비스 내부에 통합 (베타 운영 안정성)
- ☐ Postman 컬렉션 v1 작성 + P2와 공유
- ☐ Docker로 FastAPI 패키징 (P2가 Spring Boot에서 호출 가능하도록)
- ☐ ML 서비스 ↔ Spring Boot 인터페이스 명세 v1 P2와 확정

입력

- drills_seed_v5_1.json, drill_recommender_v5.py (기존)
- model_router.py, clarification_loop.py, label_postprocess.py (기존)
- P2의 DB 스키마 v1 (entries, feedbacks 구조 확정)

산출물

- FastAPI 서비스 (Dockerfile 포함)
- quota_manager.py 모듈
- api_spec_v1.md (ML ↔ Spring Boot 명세)
- Postman 컬렉션 v1

검증 기준

- FastAPI /label로 50문장 호출 시 모두 정상 응답
- FastAPI /recommend로 7개 시나리오 호출 시 적정 드릴 반환
- quota_manager 단위 테스트 통과 (일일 10건, 분당 2건 차단 확인)
- P2가 Spring Boot에서 FastAPI 호출 성공

의존성

- P2의 DB 스키마 (entries 테이블 구조)

W3 (3주차) — ★ 1차 데모

작업 목록 (총 10~11시간)

- ☐ scenarios_v1.json 작성: 7개 prototype 시나리오 + 짧은(<20자)/중간/긴(>80자) 입력 각 1개씩
- ☐ 합성 데이터 sanity check 스크립트 (각 시나리오 → 추천 결과 자동 검증)
- ☐ 누적 카드 카피 자동 생성 완성 (1회·3회·7회·15회+ 4단계 분기)
- ☐ clarification_options() 함수 (사용자 측 보강 옵션): confidence 0.3~0.6 시 객관식 6개 + 건너뛰기
- ☐ 위기 신호 감지 시 안내 카드 응답 (전문기관 정보 포함)
- ☐ 1차 데모 리허설: 입력 → 라벨링 → 정밀화 → 추천 → 평가 → 누적 카드 E2E 시나리오 3개
- ☐ 발표용 데모 시나리오 3개 스크립트 (각 90초)
- ☐ 데모 영상 1차 컷 녹화 (스크린 레코딩 + 음성)
- ☐ W3 마일스톤 점검 회의 진행

입력

- P2의 Spring Boot CRUD API (entries 저장, recommend 호출)
- P3의 Stage 1 입력 + 추천 화면 v1
- P4의 누적 카드 컴포넌트 v1

산출물

- scenarios_v1.json (7개 + 길이 변형 3개 = 10개 시나리오)
- sanity_check.py 스크립트
- clarification_options 함수 (drill_recommender_v5.py에 추가)
- 위기 신호 응답 처리 코드
- 데모 영상 1차 컷
- W3 마일스톤 보고서 (전 팀에)

검증 기준 — 가장 엄격

- 시나리오 7개 모두 입력 → 예상 dominant 패턴 top-3 안에 들어옴
- E2E 흐름 (입력 → 추천 → 평가) 30초 이내 완료
- 위기 신호 시나리오에서 일반 드릴 추천 안 됨
- 데모 영상 5분 이내 작동

위험

- Spring Boot ↔ FastAPI 통합 버그 → W3 첫날 30분 합동 디버깅 슬롯 확보
- LLM 응답 지연 → 데모용 캐시 시나리오 1개 사전 준비

W4 (4주차) — 시험주차

시험주차이므로 시간 50% 수준. 신기능 추가 금지, 안정화·문서화만.

작업 목록 (총 5~6시간)

- ☐ logreg_features.md 작성: 59차원 피쳐 정의 문서화
- ☐ 합성 데이터 생성 스크립트 사전 작성 (W5에 즉시 학습 가능하도록)
- ☐ scenarios_v1.json 기반 회귀 테스트 자동화 (CI 진입 검토)
- ☐ P5와 베타 안내문·동의서 최종 확정
- ☐ W5 베타 모집 launch plan 작성
- ☐ W3에서 발견된 버그 모두 fix

산출물

- logreg_features.md (59차원 피쳐 명세서)
- generate_synthetic_data.py (사전 작성)
- 베타 모집 launch plan

검증 기준

- 회귀 테스트 7개 시나리오 모두 통과
- W3 발견 버그 0건 남아있음

W5 (5주차) — 피크 주차 ★

작업 목록 (총 12~13시간)

- ☐ 합성 데이터 200건 생성 (entries 50 + feedbacks 200 시뮬레이션)
- ☐ drill_recommender_v1.py 작성: Logistic Regression 학습 모듈
- ☐ 59차원 피쳐 추출 함수 (사용자 11 + 드릴 23 + 카테고리 6 + 상호작용 17 + intensity_short + duration)
- ☐ sklearn LogisticRegression(class_weight='balanced', penalty='l2', solver='lbfgs')
- ☐ 5-fold CV로 AUC 측정 — 목표 AUC > 0.65
- ☐ AUC 미달 시 피쳐 재설계 (2-3 iteration 반복)
- ☐ 사용자 baseline 추출 함수 (1주 데이터 누적 후 평균 분포)
- ☐ 거부 피드백 → 다음 prior 반영 로직 (전략 5)
- ☐ 베타 8~12명 모집 시작 (학과 커뮤니티, 동아리, 지인)
- ☐ 베타 사용자 개별 안내 + 시스템 사용법 30분 영상 녹화
- ☐ 베타 사용자 GAD-7/PHQ-9 사전 시행 (Google Form)

입력

- logreg_features.md (W4 산출물)
- generate_synthetic_data.py (W4 산출물)
- P2의 베타 운영 환경 (Render/Railway 배포)

산출물

- drill_recommender_v1.py (LogReg 학습 + 추론 함수)
- model_v1.pkl (학습된 모델 파일)
- synthetic_auc_report.md (합성 데이터 AUC 결과)
- 베타 사용자 가이드 영상 (30분)
- GAD-7/PHQ-9 사전 데이터 (Google Form 응답)

검증 기준

- 합성 데이터 5-fold CV AUC ≥ 0.65
- 베타 8명 이상 활성 사용자 확보
- 사전 척도 응답률 100%

위험

- AUC < 0.65 → v0 단독으로 W6 진행 + 발표에서 정직 보고

- 베타 8명 미달 → 지인 직접 1:1 영업으로 보강

W6 (6주차) — 피크 주차 ★

작업 목록 (총 12~13시간)

- ☐ 베타 1주 실사용 데이터 일일 점검 (매일 30분)
- ☐ 이슈 트리아지 (긴급 버그 P2와 즉시 대응)
- ☐ drill_recommender_v1 베타 데이터로 재학습 (200+ 실제 데이터)
- ☐ v0 vs v1 helpful_rate ablation
- ☐ 검증 분석 1: affinity weight vs helpful_rate Spearman 상관
- ☐ 검증 분석 2: 패턴 점수 × PHQ-9/GAD-7 수렴 상관
- ☐ 검증 분석 3: v0 vs v1 AUC 비교
- ☐ 검증 분석 4: Stage 1/2/3 수락률 + '아닌 것 같아요' 클릭률
- ☐ validation_report.md v2 완성 (4종 분석 결과 + 해석)
- ☐ 베타 종료 시 GAD-7/PHQ-9 사후 시행
- ☐ 사전·사후 척도 paired t-test (statistical significance)

산출물

- model_v1_beta.pkl (베타 데이터 재학습 모델)
- validation_report.md v2 (4종 검증 결과 + 통계 분석)
- 사전·사후 척도 비교 분석
- 발표 자료용 그래프 5장 (matplotlib/seaborn)

검증 기준

- validation_report.md에 모든 분석 결과 수치 명시
- v1 AUC > v0 baseline (음수도 정직 보고)
- Spearman r 또는 통계 유의성 명시 (p-value)

W7 (7주차) — 발표

작업 목록 (총 10~11시간)

- ☐ 발표 슬라이드 v1 작성 (40~60장):
- ☐ 도입: 문제 정의 + 4가지 차별점
- ☐ 시스템 구조: v0 → v1 → v2 진화 다이어그램
- ☐ 방법론: LLM 라벨링 + 시스템 후처리 + 객관식 정밀화
- ☐ W1 검증: Cohen's κ 3개 + 해석
- ☐ 추가 기능 3개: 누적 카드, 패턴 그래프, 효과 비교 카드
- ☐ 베타 결과: ablation, 수렴 상관, 사전·사후 척도, 정성 피드백
- ☐ 한계 인정 + v2 로드맵 (비슷했던 날, Pro 계정)
- ☐ BM 및 확장 가능성: 사용자 100~500명 무료 한도 안에서 운영 가능 근거
- ☐ 발표 리허설 2회 진행 (수·금)
- ☐ Q&A 답변 7개 준비 (보고서 부록 C 기반)
- ☐ 데모 영상 5분 최종 컷 (P3 라이브 시연 백업용)
- ☐ 발표 당일 진행 + Q&A 응답

입력

- validation_report.md v2 (W6 산출물)
- P5의 정성 인터뷰 자료 + 사용자 인용
- P4의 발표용 차트 캡처
- P2의 시스템 아키텍처 다이어그램
- P3의 라이브 데모 환경

산출물

- 발표 슬라이드 (Google Slides 또는 Keynote)
- Q&A 답변 시나리오
- 데모 영상 5분
- 발표 당일 자료 (백업 USB 포함)

검증 기준

- 리허설 2회 진행, 발표 시간 정해진 시간 내
- Q&A 답변 7개 숙지 (다른 팀원이 임의 질문 시 답변 가능)
- 데모 영상 5분 이내 + 모든 핵심 기능 시연

P1.4 핵심 위험 — 단일 실패점 3개

P1의 작업 중 다음 3가지는 다른 팀원 전체에 영향. 사전 대비 필수.

실패점	영향	사전 대비
W2 FastAPI 골격 못 굴림	W3 데모 불가능 → 모든 후속 일정 무너짐	W1 종료 시 LLM API sanity check 완료
W5 LogReg AUC 미달	v1 발표 불가, ablation 못 함	W4 시험주차에 합성 데이터 코드 사전 작성
W6 베타 8명 미달	검증 데이터 부족, ablation 약함	W4에 베타 안내문 완성, W5 첫날 모집 시작

P1.5 P1 산출물 최종 체크리스트

W1

- ☐ labeling_results_v1.json
- ☐ labeling_audit_v1.md
- ☐ labeling_guide_cases.md (5~10 케이스)
- ☐ validation_report.md v1 (κ 3개)

W2

- ☐ FastAPI 서비스 (Docker)
- ☐ quota_manager.py
- ☐ api_spec_v1.md

W3

- ☐ scenarios_v1.json
- ☐ clarification_options 함수
- ☐ 데모 영상 1차 컷

W4

- ☐ logreg_features.md
- ☐ generate_synthetic_data.py

W5

- ☐ drill_recommender_v1.py
- ☐ model_v1.pkl
- ☐ 베타 사용자 가이드 영상

W6

- ☐ model_v1_beta.pkl
- ☐ validation_report.md v2 (4종 검증)
- ☐ 사전·사후 척도 비교

W7

- ☐ 발표 슬라이드
- ☐ Q&A 답변 시나리오
- ☐ 데모 영상 5분

P2 — 백엔드

P2.1 역할 정의

P2는 Spring Boot 기반 백엔드 API 서버의 단독 책임자다. DB 설계, 인증, P1의 FastAPI ML 서비스 호출, quota 관리 미들웨어, 배포 환경까지 담당한다. 베타 운영 중 시스템 가동률을 보장하는 마지막 보루다.

P2의 산출물이 안정적이지 않으면 베타 사용자가 입력 자체를 못 한다. W3 1차 데모와 W6 베타 운영 1주가 가장 중요한 시기.

P2.2 사용 도구 상세

도구	용도	수준
Java 17 + Spring Boot 3.x	API 서버, 비즈니스 로직	필수
Spring Security	인증 (학부 단순 토큰 인증)	필수
Spring Data JPA + Hibernate	ORM, 마이그레이션	필수
PostgreSQL 15+	주 DB, JSONB 활용 (LLM 결과 저장)	필수
Docker + docker-compose	로컬 개발 환경	필수
Postman	API 테스트, 컬렉션 관리	필수
Render 또는 Railway	배포 환경 (무료 tier)	필수
Flyway 또는 Liquibase	DB 마이그레이션	권장

P2.3 주차별 작업

W1 (1주차)

작업 목록 (총 8~9시간)

- ☐ Spring Boot 3.x 프로젝트 초기 셋업 (Spring Initializr)
- ☐ 의존성 추가: Web, Security, Data JPA, PostgreSQL Driver, Validation
- ☐ docker-compose.yml 작성: PostgreSQL 15 + 앱 서버
- ☐ 로컬 환경에서 'Hello World' API 동작 확인

- ☐ DB 스키마 v1 작성 (4개 테이블)
- ☐ Flyway 마이그레이션 셋업
- ☐ ML 서비스 인터페이스 명세 P1과 합의 (REST API 형식)
- ☐ GitHub repo 셋업 + 첫 commit (모든 팀원 access)
- ☐ Kickoff 회의 참석 + DB 설계 발표

입력

- 컨텍스트 문서의 백엔드 기술 스택 정의
- P1과의 API 인터페이스 합의

산출물

- Spring Boot 프로젝트 (GitHub commit)
- docker-compose.yml
- db_schema_v1.sql (또는 Flyway 마이그레이션 파일)
- api_spec_v1.md 초안

DB 스키마 v1 상세

```
-- users 테이블
CREATE TABLE users (
  id          BIGSERIAL PRIMARY KEY,
  email       VARCHAR(255) UNIQUE NOT NULL,
  display_name VARCHAR(100),
  created_at  TIMESTAMP DEFAULT NOW()
);
```

```
-- entries 테이블
CREATE TABLE entries (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT REFERENCES users(id),
  text        TEXT NOT NULL,
  self_condition INT, -- 1~5
  llm_result  JSONB, -- LLM 라벨링 결과 전체
  created_at  TIMESTAMP DEFAULT NOW()
);
CREATE INDEX idx_entries_user_created ON entries(user_id, created_at);
```

```
-- feedbacks 테이블
CREATE TABLE feedbacks (
  id          BIGSERIAL PRIMARY KEY,
```

```
entry_id    BIGINT REFERENCES entries(id),
drill_id    VARCHAR(10) NOT NULL, -- D01~D71
label       VARCHAR(20), -- 'helpful' or 'not_helpful'
created_at  TIMESTAMP DEFAULT NOW()
);
```

```
-- user_preferences 테이블
CREATE TABLE user_preferences (
  user_id    BIGINT PRIMARY KEY REFERENCES users(id),
  ask_sleep  BOOLEAN DEFAULT NULL, -- null/true/false
  ask_activity BOOLEAN DEFAULT NULL,
  ask_meal   BOOLEAN DEFAULT NULL,
  ask_caffeine BOOLEAN DEFAULT FALSE, -- 카페인 옵션
  declined_at TIMESTAMP, -- 영구 거부 시
  updated_at TIMESTAMP DEFAULT NOW()
);
```

검증 기준

- 로컬에서 Spring Boot 앱 정상 기동 + DB 연결 확인
- 4개 테이블 생성 + 외래 키 정상 동작
- api_spec_v1.md에 최소 6개 엔드포인트 명세 (P1과 합의)

W2 (2주차)

작업 목록 (총 10~11시간)

- ☐ POST /entries 엔드포인트: 텍스트 입력 → ML 호출 → 결과 저장
- ☐ GET /entries/{userId}/recent?days=N 엔드포인트 (누적 카드용)
- ☐ POST /feedbacks 엔드포인트: 드릴 평가 저장
- ☐ GET /preferences/{userId} + PATCH /preferences/{userId} (점진 입력 상태)
- ☐ ML 서비스 호출 클라이언트 (RestTemplate or WebClient)
- ☐ Spring Security 단순 토큰 인증 (학부용)
- ☐ 사용자당 일일 한도 미들웨어 (DAILY_LIMIT=10 차단)
- ☐ 에러 핸들링 (@ControllerAdvice + 표준 에러 응답)
- ☐ Postman 컬렉션 v1 완성 + 팀원 공유
- ☐ P3와 API 연동 테스트 (CORS 설정 포함)

Postman 컬렉션 v1 (필수 엔드포인트)

- POST /api/auth/signup — 회원가입
- POST /api/auth/login — 로그인 (토큰 발급)
- POST /api/entries — 한 줄 입력 (텍스트 + condition)
- GET /api/entries/me/recent — 본인 최근 N일 입력 목록
- POST /api/feedbacks — 드릴 평가
- GET /api/preferences/me — 본인 점진 입력 상태
- PATCH /api/preferences/me — 옵션 수락/거부

산출물

- Spring Boot CRUD 완성
- ML 호출 클라이언트
- Postman 컬렉션 v1
- CORS 설정 (P3 도메인 허용)

검증 기준

- 7개 엔드포인트 모두 Postman으로 정상 동작
- 일일 한도 11번째 호출 시 429 응답
- P3 프론트에서 직접 API 호출 성공

W3 (3주차) — 1차 데모 지원

작업 목록 (총 10~11시간)

- ☐ Rate limit 추가: 분당 2건, 시간당 8건
- ☐ 위기 신호 처리: ML 응답이 CRISIS_DETECTED면 전문기관 안내 카드 반환
- ☐ GET /users/{userId}/baseline 엔드포인트 (사용자 평균 패턴 분포)
- ☐ DB 인덱싱 최적화 (entries.user_id, feedbacks.entry_id)
- ☐ 로그 시스템 (Logback) + 로그 레벨별 필터링
- ☐ 1차 데모 리허설 참여 (P1과 합동)
- ☐ W3 데모 중 발생한 API 버그 즉시 fix

위기 신호 처리 코드 예시

```
// EntryController.java
if (mlResponse.isCrisisDetected()) {
    return CrisisResponseCard.of(
        "지금 많이 힘드신 것 같아요.",
        List.of(
            CrisisResource.of("자살예방상담", "1393"),
            CrisisResource.of("청소년상담", "1388")
        )
    );
}
```

검증 기준

- rate limit 발동 시 사용자에게 친절한 메시지 + 다음 가능 시각 표시
- 위기 시나리오 (S016 '그냥 사라지고 싶다') 입력 시 일반 드릴 추천 차단
- DB 인덱스 적용 후 조회 응답 100ms 이내

W4 (4주차) — 시험주차

작업 목록 (총 5~6시간)

- ☐ DB 마이그레이션 + 백업 전략
- ☐ 토큰 사용량 로깅 테이블 추가 (llm_usage_logs)
- ☐ Render 또는 Railway 배포 환경 셋업
- ☐ 환경 변수 분리 (dev / prod profile)
- ☐ 배포 후 prod 환경에서 모든 엔드포인트 sanity check

llm_usage_logs 스키마

```
CREATE TABLE llm_usage_logs (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT REFERENCES users(id),
  timestamp   TIMESTAMP DEFAULT NOW(),
  model_name  VARCHAR(50), -- 어떤 tier
  input_tokens INT,
  output_tokens INT,
  estimated_cost DECIMAL(10, 6),
  purpose     VARCHAR(30) -- 'label_primary' or 'clarification'
);
```

산출물

- 배포된 백엔드 (prod URL)
- 환경 분리 설정
- 토큰 로깅 테이블 생성

검증 기준

- prod URL에서 모든 엔드포인트 정상 응답
- 환경 변수 .env가 git에 안 올라감

W5 (5주차)

작업 목록 (총 9~10시간)

- ☐ 베타 운영 모니터링 대시보드 (간단 admin 화면)
- ☐ 토큰 사용량 일일 집계 스크립트 (cron 또는 수동)
- ☐ 비정상 패턴 알림 (특정 사용자 일일 한도 80% 도달 시 Discord webhook)
- ☐ Production DB 자동 백업 (Render 또는 외부 storage)
- ☐ API 응답 시간 모니터링 (Spring Boot Actuator)
- ☐ 베타 가입 시 user_preferences 자동 생성

모니터링 대시보드 (Thymeleaf 또는 별도 React 페이지)

- 일일 활성 사용자 수
- 일일 입력 건수 (합계 + 사용자별 평균)
- LLM 토큰 사용량 (Primary tier vs Light tier)
- 최근 7일 추세 그래프

검증 기준

- 대시보드 페이지가 P1 admin 계정으로 접근 가능
- 일일 한도 80% 도달 시 Discord 알림 작동 확인

W6 (6주차) — 베타 운영

작업 목록 (총 9~10시간)

- ☐ 메타 카드 백엔드 API (GET /users/{id}/weekly-summary)
- ☐ 주간 요약 SQL 쿼리 작성 (패턴 빈도, 시간대, 도움됐던 드릴)
- ☐ 베타 운영 중 버그 즉시 대응 (Slack/Discord 24시간 대응)
- ☐ 토큰 사용량 주간 집계 → P1에게 발표 자료용 통계 전달
- ☐ 일일 데이터 백업 모니터링

주간 요약 쿼리 예시 (PostgreSQL)

```
SELECT
  jsonb_object_keys(e.llm_result->'patterns') AS pattern,
  COUNT(*) FILTER (
    WHERE (e.llm_result->'patterns'->>pattern)::float >= 0.5
  ) AS strong_count
FROM entries e
WHERE e.user_id = :userId
  AND e.created_at >= NOW() - INTERVAL '7 days'
GROUP BY pattern;
```

산출물

- 주간 요약 API (메타 카드용)
- 주간 토큰 사용량 보고서 (발표 자료 input)

검증 기준

- 주간 요약 API 응답 시간 200ms 이내
- 베타 운영 중 시스템 가동률 99% 이상

W7 (7주차) — 발표 준비

작업 목록 (총 6~7시간)

- ☐ 시스템 아키텍처 다이어그램 작성 (Excalidraw 또는 draw.io)
- ☐ 베타 운영 통계 슬라이드 데이터:
- ☐ 일일 활성 사용자 수, 평균 입력/사용자, 시스템 가동률
- ☐ 토큰 사용량 (무료 한도의 X%)
- ☐ Stage 1/2/3 수락률 데이터 (P1·P3와 협업)
- ☐ 발표 후 코드 정리 + README 작성 (오픈소스 공개 대비)
- ☐ 발표 자료 백엔드 섹션 슬라이드 4~5장 P1에게 전달

아키텍처 다이어그램 구성

- Next.js 클라이언트 (P3) ← REST API → Spring Boot
- Spring Boot → FastAPI ML 서비스 (P1)
- FastAPI → Gemini API (Primary tier + Light tier)
- Spring Boot ↔ PostgreSQL
- 관찰: rate limit, quota_manager, 위기 신호 분기

산출물

- 아키텍처 다이어그램 (PNG/SVG)
- 베타 통계 데이터 (CSV 또는 표)
- README.md (프로젝트 소개)

P2.4 핵심 위험

위험	영향	사전 대비
W2 ML 호출 통합 실패	P3 프론트 작업 차단	W1 마지막 날 mock API라도 P3에 제공
배포 환경 불안정	베타 운영 중단	W4 시험주차에 prod 환경 안정성 확인 + 백업 환경
DB 백업 실패	사용자 데이터 손실 시 복구 불가	W4부터 일일 백업 자동화 + 검증
rate limit 너무 강함	정당한 사용자 차단	W3 데모에서 베타급 사용 패턴 테스트

P2.5 P2 산출물 최종 체크리스트

W1

- ☐ Spring Boot 프로젝트
- ☐ DB 스키마 v1 (4개 테이블)
- ☐ api_spec_v1.md 초안

W2

- ☐ CRUD API 7개 엔드포인트
- ☐ ML 호출 클라이언트
- ☐ 일일 한도 미들웨어
- ☐ Postman 컬렉션 v1

W3

- ☐ Rate limit (분/시간/일)
- ☐ 위기 신호 처리 분기
- ☐ baseline 조회 API

W4

- ☐ prod 배포 환경
- ☐ llm_usage_logs 테이블

- ☐ 자동 백업 셋업

W5

- ☐ 모니터링 대시보드
- ☐ 토큰 집계 스크립트
- ☐ Discord 알림

W6

- ☐ 주간 요약 API
- ☐ 주간 토큰 보고서

W7

- ☐ 아키텍처 다이어그램
- ☐ 베타 통계
- ☐ README.md

